Latent Manifold Estimation for House Pricing

Paper-aligned Implementation with Full Technical Notes

Capstone Group

November 19, 2025

Abstract

We describe a paper-aligned Latent Manifold Estimation (LME) pipeline for residential pricing. The target is modeled as the sum of an *intrinsic* component predicted from tabular home attributes via a neural network and a *desirability* field that captures neighborhood effects using a kernel surface over geospatial coordinates. We give precise notation, derive the quadratic problem for the desirability vector, explain the matrix-free Conjugate Gradient (CG) solve with early stopping, detail the residual training of the intrinsic network with spatial validation, and map each mathematical object to the corresponding functions/classes in our codebase. Hyperparameters, splits, metrics, and engineering choices are documented for reproducibility.

1 Intuition and model overview

Homes with similar structural attributes can still sell for very different prices because of **neighborhood desirability**: school clusters, street micro-amenities, noise, safety, walkability, etc. We therefore decompose the log-target into an intrinsic (structure-driven) term and a spatially-smoothed desirability term.

For property $i \in \{1, \dots, N\}$ we observe $\boldsymbol{x}_i^{(\text{param})} \in \mathbb{R}^d$ (tabular features), $\boldsymbol{s}_i \in \mathbb{R}^2$ (lat/lon), nonnegative weight w_i, and log-target y_i. We posit the additive model

$$y_i = m_i + h_i + \varepsilon_i, \quad m_i = G(\boldsymbol{x}_i^{(\text{param})}; \boldsymbol{W}), \quad h_i = H(\boldsymbol{s}_i; \boldsymbol{D}), \tag{1}$$

where G is a BN+ReLU MLP parameterized by $\boldsymbol{W}$ and H is a nonparametric spatial smoother controlled by $\boldsymbol{D} \in \mathbb{R}^N$. The vector $\boldsymbol{D}$ holds one scalar "desirability" value per *train* home; at inference, a new location interpolates h_* from its neighbors' d_j values.

2 Variable glossary (symbols $\leftrightarrow$ code)

3 Geometry: projecting lat/lon and standardization

We convert degrees to a local metric plane centered at the train mean latitude $\bar{\phi}$:

$$x_i = R(\lambda_i - \bar{\lambda}) \cos \bar{\phi}, \qquad y_i = R(\phi_i - \bar{\phi}), \qquad R = 6.371 \times 10^6 \,\text{m}. \tag{2}$$

The surface coordinates $[y_i, x_i]$ are standardized by train mean/std (so KDTree distances are isotropic). **Code:** `latlon_to_xy_meters`, `standardize_by_train`.

Table 1: Symbols and their roles with code references.

Symbol	Code handle	Meaning / Effect
$\boldsymbol{x}_i^{(\text{param})}$	`X_tr[i]`	Standardized param features for home i; input to MLP.
$\boldsymbol{s}_i = [\text{lat}, \text{lon}]_i$	`S_tr_std[i]`	Standardized meters (Eq. (2)); drives spatial neighbors.
y_i	`y_tr[i]`	Log target: log(PPSQFT) if sqft available, else log(price).
w_i	`w_tr[i]`	Sample weight (upweights higher PPSQFT quantiles).
$\boldsymbol{W}$	`model.state_dict()`	MLP parameters of the intrinsic function G.
$\boldsymbol{D}$	`trainer.D`	Desirability vector; one scalar per train row; produces the surface via neighbors.
K	`hp.K`	# neighbors in KDTree; larger K = smoother/less variance, more bias.
q	`hp.q`	Kernel sharpness; larger q decays weights faster, reducing spatial bleed.
r	`hp.reg_r`	Ridge on D; higher r shrinks D to zero, shifting burden to MLP.
λ	`hp.lap_lambda`	Laplacian smoothing weight; controls spatial coherence of D.
$\mathbf{L}$	`LaplacianOp`	Matrix-free Laplacian operator (Eq. (7)).
$\mathcal{N}(i)$	KDTree query	The K nearest neighbors of i in standardized meters.
w_{ij}	`KernelSurface`	Adaptive kernel weights (Eq. (3)); $\sum_{j\in\mathcal{N}(i)} w_{ij} = 1$.
U_i	implicit	Sparse vector with $(U_i)_j = w_{ij}$ if $j \in \mathcal{N}(i)$ else 0.

4 Kernel surface and interpolation

For each train point i, KDTree returns neighbors $\mathcal{N}(i)$ and distances d_{ij}. We use an *adaptive* bandwidth $\sigma_i^2 = \text{median}\{d_{ij}^2\} + 10^{-12}$ and define

$$w_{ij} \propto \exp\left(-\frac{q\, d_{ij}^2}{2\sigma_i^2}\right), \qquad \sum_{j\in\mathcal{N}(i)} w_{ij} = 1. \tag{3}$$

The surface values are

$$h_i = \sum_{j\in\mathcal{N}(i)} w_{ij}\, d_j \quad \text{(train)}, \qquad h(\boldsymbol{s}_*; \boldsymbol{D}) = \sum_{j\in\mathcal{N}(*)} \tilde{w}_{*j}\, d_j \quad \text{(inference)}. \tag{4}$$

Code: `KernelSurface.build_U_list`, `KernelSurface.interpolate_one`.

5 Objective and EM decomposition

We minimize the *energy*

$$\mathcal{E}(\boldsymbol{W}, \boldsymbol{D}) = \frac{1}{2}\sum_{i=1}^{N} w_i\big(y_i - m_i - h_i\big)^2 + \frac{r}{2}\,\|\boldsymbol{D}\|_2^2 + \frac{\lambda}{2}\,\boldsymbol{D}^\top \mathbf{L}\,\boldsymbol{D}. \tag{5}$$

With G fixed, $\mathcal{E}$ is quadratic in $\boldsymbol{D}$; with $\boldsymbol{D}$ fixed, it is a standard weighted MSE for the MLP trained on *residual* targets.

E-step: solving for $\boldsymbol{D}$

Let U_i be the sparse weight vector for point i (Eq. (3)). The normal equations are

$$\Big(r\,\mathbf{I} + \sum_{i=1}^{N} w_i\, U_i U_i^\top + \lambda \mathbf{L}\Big)\, \boldsymbol{D} = \sum_{i=1}^{N} w_i\, (y_i - m_i)\, U_i. \tag{6}$$

We solve (6) using matrix-free Conjugate Gradient: given $\boldsymbol{v}$, we compute $A\boldsymbol{v} = r\boldsymbol{v} + \sum_i w_i (U_i^\top \boldsymbol{v}) U_i + \lambda \mathbf{L}\boldsymbol{v}$ in $O(NK)$ time without forming A. Early stopping uses (i) relative residual threshold and (ii) objective plateau patience. Finally we apply a *gauge correction* $\boldsymbol{D} \leftarrow \boldsymbol{D} - \frac{1}{N}\sum_i d_i$ to keep the decomposition $m + h$ identifiable. **Code:** `LMETrainer._update_D`, `cg_solve_qp`.

What each term does. $r\mathbf{I}$ shrinks D to stabilize ill-conditioned neighborhoods; $\sum_i w_i U_i U_i^\top$ enforces that D agrees with (kernel-averaged) residuals at each point; $\lambda \mathbf{L}$ encourages spatial smoothness (neighbors should have similar desirability).

M-step: training G on residuals

With $\boldsymbol{D}$ fixed, we form residual targets $y_i^{(\text{res})} = y_i - h_i$ and train the MLP with AdamW (learning rate warmup + cosine schedule) and an *inner spatial validation* split for early stopping. **Code:** `LMETrainer._mstep`.

6 Graph Laplacian (optional)

Using the same KDTree we define

$$(\mathbf{L}\boldsymbol{v})_i = \sum_{j \in \mathcal{N}_L(i)} a_{ij}\,(v_i - v_j), \qquad a_{ij} \propto \exp\!\left(-\frac{d_{ij}^2}{2\tilde{\sigma}_i^2}\right), \tag{7}$$

so that $\boldsymbol{D}^\top \mathbf{L} \boldsymbol{D}$ penalizes roughness of D over the neighbor graph. **Code:** `LaplacianOp.apply`.

7 Data pipeline and splits

Preprocessing: load raw CSV; clean/engineer features; prepare matrix X, target y, and spatial columns. Standardize X and surface coords by train statistics only.

Spatial TRAIN/TEST: coarse 100×100 grid over degrees to pick disjoint regions (reduces leakage). **Inner VAL:** a second 60×60 grid inside TRAIN for M-step early stopping. **Code:** `DataPreprocessor`, `spatial_grid_split`, `make_inner_spatial_val`.

8 Architecture and training details

- Intrinsic network: MLP with hidden sizes (256, 128, 64, 32), BatchNorm+ReLU, dropout $p = 0.25$.

- Optimizer: AdamW, learning rate 5×10^{-4}, weight decay 5×10^{-4}, batch size 512.
- LR schedule: linear warmup to epoch E_{warm} then cosine decay within the M-step.
- CG: tol 10^{-5}, max 300 iters, patience 10; complexity per matvec $O(NK)$.
- Regularization: ridge $r = 0.05$; optional Laplacian $\lambda \in \{0, 0.02\}$.
- Target/weights: prefer $y = \log(\text{PPSQFT})$ (if SQFT available); upweight median and 90th-percentile PPSQFT by $\times 1.2$ and $\times 1.6$.

9 Function-to-equation map (implementation guide)

Table 2: Where each equation lives in code.

Equation	Code function / class
Additive model (1)	`LMETrainer.predict`, `IntrinsicPriceNet.forward`, `KernelSurface.interpolate`
Projection/standardize (2)	`latlon_to_xy_meters`, `standardize_by_train`
Kernel weights (3)	`KernelSurface._adapt_weights`, `KernelSurface.build_U_list`
Surface interp (4)	`KernelSurface.interpolate_one`, `LMETrainer._compute_h`
Energy (5)	reported in `trainer.fit()` (`em_losses`)
Normal equations (6)	`LMETrainer._update_D` + `cg_solve_qp`
Laplacian (7)	`LaplacianOp.apply`
Residual targets $y - h$	`LMETrainer._mstep`

10 Hyperparameters used

Table 3: Default hyperparameters for reported runs.

Group	Values
Neighbors	$K = 40$, $q = 1.0$ (adaptive bandwidth)
Regularization	$r = 0.05$, $\lambda = 0.02$ (optional)
EM	outer iters $T = 6$; warmup epochs = 8; M-step epochs = 12; patience = 10
Optimizer	AdamW; LR 5×10^{-4}; weight decay 5×10^{-4}; batch = 512
Network	hidden = $(256, 128, 64, 32)$; dropout = 0.25
CG	tol = 10^{-5}; max iters = 300; patience = 10
Splits	Spatial test = 20%; inner spatial VAL = 12% of TRAIN

11 Practical effects of key knobs

- K: higher K increases bias (smoother h), reduces variance; too low K yields noisy D.
- q: larger q shrinks kernel support; prevents spatial bleed but can under-smooth.

- r: stronger ridge shrinks $\boldsymbol{D}$ toward zero; the MLP must explain more via m.
- λ: increases spatial coherence; mitigates speckle in D when data are sparse.
- **Warmup/cosine**: stabilizes M-step training against abrupt LR jumps after E-step changes.
- **VAL split**: prevents overfitting the residuals to inner-train neighborhoods.

12 Metrics and results

We report absolute-relative metrics in price (or PPSQFT) space:

$$\text{AbsRel}(i) = \frac{|\,\hat{P}_i - P_i\,|}{P_i + 10^{-12}}, \quad \text{Pct}(<\tau) = \frac{1}{M}\sum_i \mathbf{1}\{\text{AbsRel}(i) < \tau\}, \quad \text{MedianAbsRel} = \text{median}(\text{AbsRel}). \tag{8}$$

Comparison to paper (PPSQFT, With Loc)

Table 4: Test accuracy within price error thresholds (PPSQFT).

Algorithm	$<$ **5**%	$<$ **15**%
Nearest Neighbor (paper)	27.51	68.31
Neural Network (paper)	27.43	70.10
LME (kernel) (paper)	**29.39**	**71.70**
LME (llr) (paper)	29.67	72.06
S-LME (llr) (paper)	29.69	72.15
Our LME (this work)	**18.10**	**50.62**

Our Train vs Test (PPSQFT)

Table 5: Paper-style metrics for our implementation.

	$<$ **5**%	$<$ **10**%	$<$ **15**%	**Median abs rel**
Train	25.54%	49.10%	67.54%	0.1022
Test	18.10%	35.45%	50.62%	0.1479

13 Failure modes and diagnostics

- **NaNs in CG:** typically from unstandardized surface, missing lat/lon, or empty neighbor lists.
- **Degenerate split:** if a spatial cell selection yields zero samples, fall back to random split.
- **Over-smoothing D:** K or λ too high; reduces local signal but inflates bias.
- **Leakage:** ensure test cells are truly disjoint from train; reuse train stats only.

14 Reproducibility checklist

- Project lat/lon with Eq. (2); standardize by train.
- Build KDTree on train; $K = 40$; adaptive kernel Eq. (3).
- EM: $T = 6$, CG tol 10^{-5}, patience 10; M-step epochs 12 with warmup+cosine; inner spatial VAL 12%.
- Metrics computed in linear price space from log predictions with clipping at ± 20.